```
' ------------------------------------------- '
' Macro Name: Auto Tool Zero                  '
' Developer: Nykon Hrytsyshyn                 '
' Date of release: 2025-05-27                 '
' ------------------------------------------- '

Option Explicit
Option Base 1

' ------------------------------------------- '
'       Settings for Auto Tool Zero macro     '
' ------------------------------------------- '

Const MACRO_UNITS As Integer = 0                    ' Units of the macro (0 = millimeters, 1 = inches)
Const PROBE_FEED As Double = 200.0                  ' Feedrate to use for probing (units/min)
Const FINISH_PROBE_FEED As Double = 10.0            ' Feedrate to use for finish probing (units/min)

Const PLATE_OFFSET As Double = 0.0                  ' Offset for the touch plate (units) (Formula: ProbePos + PlateOffset = Z-axis position)
Const SAFE_MARGIN As Double = 1.0                   ' Safe margin to return the tool from certain position (units) (Formula: CurrentPos + SafeMargin = Safe Position)
Const Z_UPPER_BOUND As Double = 0.0                 ' Upper Z-axis boundary in machine coordinates (units)
Const Z_LOWER_BOUND As Double = -60.0               ' Lower Z-axis boundary in machine coordinates (units)

Const ENABLE_DEBUG_MODE As Boolean = True           ' Set to True to enable debug messages (Useful for development and testing)
Const ENABLE_RETURN_TO_SAVED_POS As Boolean = True  ' Set to True to enable returning to the saved position functionality
Const ENABLE_PROBE_CURR_POS As Boolean = True       ' Set to True to enable probing from the current position functionality

' ------------------------------------------------------------------------------ '
'   __          __     _____  _   _ _____ _   _  _____   __________  _   _ _____    '
'   \ \        / /\   |  __ \| \ | |_   _| \ | |/ ____| |___  / __ \| \ | |  ____|   '
'    \ \  /\  / /  \  | |__) |  \| | | | |  \| | |  __     / / |  | |  \| | |__      '
'     \ \/  \/ / /\ \ |  _  /| . ` | | | | . ` | | |_ |   / /| |  | | . ` |  __|     '
'      \  /\  / ____ \| | \ \| |\  |_| |_| |\  | |__| |  / /_| |__| | |\  | |____    '
'       \/  \/_/    \_\_|  \_\_| \_|_____|_| \_|\_____| /_____\____/|_| \_|______|   '
'                                                                                    '
' ------------------------------------------------------------------------------ '
' Next constants are Mach3-specific and are used for the macro. The macro is         '
' designed to work only with Mach3 and uses its API to control the machine.          '
' Unfortunately, the documentation of the current functions did not stay current, so '
' the macro may not work with the certain versions of Mach3. The manual was mostly   '
' written by volunteers, so it is recommended to familiarise yourself with the       '
' macro and test it before using it.                                                 '
'                                                                                    '
' See Mach3 "Macro Programming Reference Manual" for more information:               '
' https://machsupport.com/wp-content/uploads/2013/02/Mach3_V3.x_Macro_Prog_Ref.pdf   '
' ------------------------------------------------------------------------------ '

Const SGN_PROBE As Integer = 22             ' Signal number for probe

Const LED_TOOL_LENGTH As Integer = 28       ' LED number for tool length compensation (G43/G49)
Const LED_COORD_ABS As Integer = 48         ' LED number for absolute distance mode (G90)
Const LED_COORD_INC As Integer = 49         ' LED number for incremental distance mode (G91)
Const LED_TOOL_CHANGING As Integer = 806    ' LED number for tool change in process
Const LED_PROBE As Integer = 825            ' LED number for probe (Diditize Input)

Const DRO_MOVE_MODE As Integer = 819        ' DRO number for move mode (0 = G00, 1 = G01)
Const DRO_FIXTURE_OFFSET_X As Integer = 830 ' DRO number for X-axis multi-function DRO
Const DRO_FIXTURE_OFFSET_Y As Integer = 831 ' DRO number for Y-axis multi-function DRO
Const DRO_FIXTURE_OFFSET_Z As Integer = 832 ' DRO number for Z-axis multi-function DRO
Const DRO_FIXTURE_OFFSET_A As Integer = 833 ' DRO number for A-axis multi-function DRO
Const DRO_FIXTURE_OFFSET_B As Integer = 834 ' DRO number for B-axis multi-function DRO
Const DRO_FIXTURE_OFFSET_C As Integer = 835 ' DRO number for C-axis multi-function DRO

Const VAR_PROBE_OFFSET_X As Integer = 2000  ' Variable number for probe offset by X-axis
Const VAR_PROBE_OFFSET_Y As Integer = 2001  ' Variable number for probe offset by Y-axis
Const VAR_PROBE_OFFSET_Z As Integer = 2002  ' Variable number for probe offset by Z-axis

' Machine message types
'
' Integer value indicating which buttons will be displayed
' in the message box
Const MSG_TYPE_OK As Integer = 0                ' OK button
```

```
Const MSG_TYPE_OK_CANCEL As Integer = 1           ' OK and Cancel buttons
Const MSG_TYPE_ABORT_RETRY_IGNORE As Integer = 2  ' Abort, Retry and Ignore buttons
Const MSG_TYPE_YES_NO_CANCEL As Integer = 3       ' Yes, No and Cancel buttons
Const MSG_TYPE_YES_NO As Integer = 4              ' Yes and No buttons
Const MSG_TYPE_RETRY_CANCEL As Integer = 5        ' Retry and Cancel buttons
Const MSG_TYPE_CANCEL_TRY_CONTINUE As Integer = 6 ' Cancel, Try Again and Continue buttons

' Machine message buttons
'
' Integer value indicating which button the user clicked
Const MSG_RETURN_OK As Integer = 1        ' OK button
Const MSG_RETURN_CANCEL As Integer = 2    ' Cancel button
Const MSG_RETURN_ABORT As Integer = 3     ' Abort button
Const MSG_RETURN_RETRY As Integer = 4     ' Retry button
Const MSG_RETURN_IGNORE As Integer = 5    ' Ignore button
Const MSG_RETURN_YES As Integer = 6       ' Yes button
Const MSG_RETURN_NO As Integer = 7        ' No button
Const MSG_RETURN_TRY As Integer = 10      ' Try Again button
Const MSG_RETURN_CONTINUE As Integer = 11 ' Continue button

' Tool parameters
'
' Integer value indicating a specific tool parameter
Const TOOL_DIAMETER As Integer = 1 ' Tool diameter
Const TOOL_Z_OFFSET As Integer = 2 ' Tool Z-axis offset
Const TOOL_X_WEAR As Integer = 3   ' Tool X-axis wear
Const TOOL_Z_WEAR As Integer = 4   ' Tool Z-axis wear

' Move Modes
'
' Integer value indicating the move mode
Const MOVE_MODE_RAPID As Integer = 0  ' Rapid move (G00)
Const MOVE_MODE_LINEAR As Integer = 1 ' Linear move (G01)

' Units of measurement
'
' Integer value indicating the units of measurement
Const UNITS_MM As Integer = 0 ' Millimeters (G21)
Const UNITS_IN As Integer = 1 ' Inches (G20)

' Coordinate Modes
'
' Integer value indicating the distance mode
Const DIST_MODE_ABS As Integer = 0 ' Absolute distance mode (G90)
Const DIST_MODE_INC As Integer = 1 ' Incremental distance mode (G91)

' Axis identifiers
'
' Each axis is represented by an integer value, which is used
' to identify the axis in the Mach3 API.
Const X_AXIS As Integer = 0
Const Y_AXIS As Integer = 1
Const Z_AXIS As Integer = 2
Const A_AXIS As Integer = 3
Const B_AXIS As Integer = 4
Const C_AXIS As Integer = 5

' ------------------------------------------- '
'        Custom macro-specific constants      '
' ------------------------------------------- '
' These constants are only  used in this macro  '
' and are not part of the Mach3 API.           '
' ------------------------------------------- '

' Message types
'
' Used for macro-specific message functions
Const MACRO_MSG_TYPE_STATUS As Integer = 0 ' Displays a message in the status bar of Mach3
Const MACRO_MSG_TYPE_DIALOG As Integer = 1 ' Displays a message in a dialog box

' Probe result keys
Const KEY_PROBE_EXIT As String = "probe_exit"
Const KEY_PROBE_POS As String = "probe_pos"

```

```
' Macro modes keys
Const KEY_NAME As String = "name"
Const KEY_DESC As String = "desc"

' ------------------------------------------- '
'         Cached variables for the macro      '
' ------------------------------------------- '
' These variables are used to store the values  '
' of the frequently used variables in the macro.'
' This is done to avoid calling the Mach3 API   '
' functions multiple times, which can slow down '
' the macro execution.                          '
' ------------------------------------------- '

' Simple implementation of the carriage return and line feed
' character sequence (CRLF) for message boxes
Dim CRLF As String

' Macro modes array
Dim MACRO_MODES(2) As Object

' Used to display the units of measurement
' in the string format (inches/mm)
Dim STR_MACRO_UNITS As String

' ------------------------------------------- '
'    Cached current parameters of the machine   '
' ------------------------------------------- '
' These variables are used to save the current  '
' parameters of the machine before running the  '
' macro.                                        '
' If the macro is canceled in the right way,    '
' the original properties will be restored.     '
' ------------------------------------------- '

Dim SAVED_MOVE_MODE As Integer      ' Saved move mode (G00/G01)
Dim SAVED_UNITS As Integer          ' Saved units of measurement (inches/mm)
Dim SAVED_EN_TOOL_LENGTH As Boolean ' Saved tool length compensation state (G43/G49)
Dim SAVED_DISTANCE_MODE As Integer  ' Saved distance mode (G90/G91)
Dim SAVED_FEED As Double            ' Saved feedrate (units/min)
Dim SAVED_Z_OFFSET As Double        ' Saved Z-axis offset for current fixture
Dim SAVED_X As Double               ' Saved X-axis machine position
Dim SAVED_Y As Double               ' Saved Y-axis machine position
Dim SAVED_Z As Double               ' Saved Z-axis machine position

' ---------------------------------------------------------------- '

Call InitCache() ' Initializes the cache for the macro
Call Main()      ' Calls the main subroutine, all logic is executed here

' ---------------------------------------------------------------- '

' Cache initialization subroutine.
'
' Initializes the cache for the macro.
Sub InitCache()
    ' Init carriage return and line feed
    CRLF = Chr$(13) & Chr$(10)

    ' Save the current units of measurement
    STR_MACRO_UNITS = TernaryIf(MACRO_UNITS = UNITS_IN, "in", "mm")

    ' Zero Tool mode
    Set MACRO_MODES(1) = CreateObject("Scripting.Dictionary")
    MACRO_MODES(1).Add KEY_NAME, "Zero Tool"
    MACRO_MODES(1).Add KEY_DESC, "Set Z-axis to zero by probing the touch plate"

    ' Tool Length Compensation mode
    Set MACRO_MODES(2) = CreateObject("Scripting.Dictionary")
    MACRO_MODES(2).Add KEY_NAME, "Tool Length Compensation"
    MACRO_MODES(2).Add KEY_DESC, "Set tool length compensation by probing the touch plate"

    ' Save current parameters of the machine
    SAVED_MOVE_MODE = GetMoveMode()
```

```
    SAVED_UNITS = GetUnits()
    SAVED_EN_TOOL_LENGTH = IsToolLengthEnabled()
    SAVED_DISTANCE_MODE = GetDistMode()
    SAVED_FEED = FeedRate()
    SAVED_Z_OFFSET = GetFixtureOffset(Z_AXIS)
    SAVED_X = GetABSPosition(X_AXIS)
    SAVED_Y = GetABSPosition(Y_AXIS)
    SAVED_Z = GetABSPosition(Z_AXIS)
End Sub

' Main subroutine.
'
' This is where the main logic of the macro is executed.
Sub Main()
    ' Check if the "E-Stop" is active
    ' If it is, exit the macro and display an error message
    If (IsEStop()) Then
        Call ExitWithError(MACRO_MSG_TYPE_DIALOG, "E-Stop is active!" & CRLF & _
            "Please reset the E-Stop before running the macro.")
    End If

    ' Debug saved parameters
    Call Debug(MACRO_MSG_TYPE_DIALOG, "Current Savings:" & CRLF & _
        "- Move intMode: " & SAVED_MOVE_MODE & " (" & TernaryIf(SAVED_MOVE_MODE = MOVE_MODE_RAPID, "G00 - Rapid)", "G01 - Linea
r)") & CRLF & _
        "- Units: " & SAVED_UNITS & " (" & TernaryIf(SAVED_UNITS = UNITS_IN, "G20 - Inches)", "G21 - Millimeters)") & CRLF & _
        "- Tool Length Compensation: " & TernaryIf(SAVED_EN_TOOL_LENGTH, "Enabled (G43)", "Disabled (G49)") & CRLF & _
        "- Distance intMode: " & SAVED_DISTANCE_MODE & " (" & TernaryIf(SAVED_DISTANCE_MODE = DIST_MODE_ABS, "G90 - Absolute)",
"G91 - Incremental)") & CRLF & _
        "- Feedrate: " & SAVED_FEED & TernaryIf(SAVED_UNITS = UNITS_IN, "in/min", "mm/min") & CRLF & _
        "- Z-Axis Offset: " & SAVED_Z_OFFSET & STR_MACRO_UNITS & CRLF & _
        "- Position in Machine Coordinates:" & CRLF & _
        "  - X: " & SAVED_X & STR_MACRO_UNITS & CRLF & _
        "  - Y: " & SAVED_Y & STR_MACRO_UNITS & CRLF & _
        "  - Z: " & SAVED_Z & STR_MACRO_UNITS)

    ' Check for probe wiring
    ' If the probe is not wired, exit the macro and display an error message
    If (IsSuchSignal(SGN_PROBE) = 0) Then
        Call ExitWithError(MACRO_MSG_TYPE_DIALOG, "Probe is not wired!" & CRLF & _
            "Please wire the probe before running the macro.")
    End If

    ' Check for probe grounding
    ' If the probe is already grounded, exit the macro and display an error message
    If (IsProbeWired()) Then
        Call ExitWithError(MACRO_MSG_TYPE_DIALOG, "Probe is already grounded!" & CRLF & _
            "Please unground the probe before running the macro.")
    End If

    ' Ask user for the mode of the operation
    Dim intMacroMode As Integer
    intMacroMode = SuggestModes()

    ' Debug selected mode
    Call Debug(MACRO_MSG_TYPE_STATUS, "Selected mode: " & intMacroMode)

    Call RunMacroMode(intMacroMode) ' Run the selected mode
    Call ExitMacro()                ' Exit the macro and restore original properties
End Sub

' ------------------------------------------- '
'               Macro modes logic             '
' ------------------------------------------- '

' This function is called to suggest the user the
' available modes of interaction with the macro.
Function SuggestModes() As Integer
    Dim strModes As String
    Dim intMode As Integer
    Dim intMacroModesSize As Integer

    intMacroModesSize = UBound(MACRO_MODES)

```

```
    ' Loop through the available modes and add them to the string
    For intMode = 1 To intMacroModesSize
        strModes = strModes & intMode & ". " & MACRO_MODES(intMode)(KEY_DESC) & TernaryIf(intMode < intMacroModesSize, CRLF, "")
    Next intMode

    ' Tell the user about available modes
    Call CancellableMsg("After pressing OK, you will be prompted to select the mode of operation." & CRLF & CRLF & _
        "Available modes:" & CRLF & strModes, _
        "Auto Tool Zero")

    ' Ask the user to select the mode
    SuggestModes = BoundedIntQuestion("Select the mode of operation (1-" & intMacroModesSize & "):", 1, intMacroModesSize)
End Function

' This subroutine is called to run the selected macro mode.
'
' It will call the appropriate subroutine based on the mode
' selected by the user.
Sub RunMacroMode(ByVal MacroMode As Integer)
    Select Case MacroMode
        Case 1
            Call ZeroZCoordinate()
        Case 2
            Call ToolLengthCompensation()
        Case Else
            Call ExitWithError(MACRO_MSG_TYPE_DIALOG, "Invalid mode selected")
    End Select
End Sub

' Sets the Z-axis to zero by probing the touch plate.
'
' It will probe the Z-axis of the current tool and save
' Z-axis position to local coordinate system. (G54-G59)
Sub ZeroZCoordinate()
    ' Propose the user to enter the offset value for Z-Axis
    ' which will be added to the probe position
    Dim dblOffset As Double
    dblOffset = DoubleQuestion("Enter the offset value for Z-Axis (" & STR_MACRO_UNITS & "):")

    ' Disable tool length compensation
    Call SetToolLengthEnabled(False)

    ' Set fixture offset to upper bound
    Call SetOEMDRO(DRO_FIXTURE_OFFSET_Z, Z_UPPER_BOUND)

    ' If ENABLE_PROBE_CURR_POS state is set to True, it will
    ' propose the user to send the machine to home position.
    ' Otherwise, it will send the machine to home position
    ' and safe Z position by default.
    '
    ' If the user presses OK, it will automatically send the
    ' machine to home position, otherwise the machine will
    ' be sent to only safe Z position.
    Dim blnAnswerSend As Boolean

    If (ENABLE_PROBE_CURR_POS) Then
        ' Propose the user to send the machine to home position
        ' or start probing from the current XY position
        blnAnswerSend = ProposeSendHome()
    Else
        blnAnswerSend = True

        ' Send the machine to home position
        Call MachMSG("Sends the machine to home position (G28)", "Auto Tool Zero", MSG_TYPE_OK)
        Call SendHome()
    End If

    ' Ask the user for a confirmation to start probing
    Dim blnAnswerProbe As Integer
    blnAnswerProbe = MachMSG("Start probing the Z-axis?" & CRLF & CRLF & _
        "Press Yes to continue or No to cancel.", _
        "Auto Tool Zero", MSG_TYPE_YES_NO)

    ' Start probing if the user pressed Yes
```

```
    If (blnAnswerProbe = MSG_RETURN_YES) Then
        Dim objProbeResult As Object
        Set objProbeResult = StartProbing()

        ' Check if the probing result was allowed by the user
        If (Not objProbeResult(KEY_PROBE_EXIT)) Then
            Dim dblProbePos As Double
            dblProbePos = roun(objProbeResult(KEY_PROBE_POS) + dblOffset + TernaryIf(blnAnswerSend, PLATE_OFFSET, 0.0))

            ' Ask the user for a confirmation to set the Z-axis offset
            Dim blnAnswerSet As Integer
            blnAnswerSet = MachMSG("Z-axis offset: " & dblProbePos & STR_MACRO_UNITS & CRLF & CRLF & _
                "Press OK to continue or Cancel to deny the offset.", _
                "Auto Tool Zero", MSG_TYPE_OK_CANCEL)

            ' Debug the answer
            Call Debug(MACRO_MSG_TYPE_STATUS, "Return message: " & blnAnswerSet)

            ' Set the Z-axis offset in the current fixture
            Call SetFixtureOffset(Z_AXIS, TernaryIf(blnAnswerSet = MSG_RETURN_OK, dblProbePos, SAVED_Z_OFFSET))
        End If

        ' Send the machine to safe Z position
        Call SendSafeZ(Z_UPPER_BOUND)
    End If
End Sub

' Sets the tool length compensation by probing the touch plate
' with two different tools.
'
' It will probe the Z-axis of the first tool and save the value,
' after that it will display the dialog message and wait for
' the user to change the tool and press Yes. By then it will
' probe the Z-axis of the new tool and calculate the difference
' between the two tool lengths. The result will be saved in the
' tool length compensation table for the second tool.
Sub ToolLengthCompensation()
    Dim intFirstTool As Integer
    intFirstTool = GetCurrentTool()

    ' Ask user for the first tool number if it is not set
    If (intFirstTool = 0) Then
        ' Ask user for the tool number
        intFirstTool = BoundedIntQuestion("Enter the first tool number (1-255):", 1, 255)
    Else
        Dim intAnswer As Integer
        intAnswer = MachMSG("Select the current instrument as the one against which the compensation will be calculated? (" & int
FirstTool & ")", _
            "Auto Tool Zero", MSG_TYPE_YES_NO)

        If (intAnswer = MSG_RETURN_NO) Then
            intFirstTool = BoundedIntQuestion("Enter the first tool number (1-255):", 1, 255)
        End If
    End If

    If (GetCurrentTool() <> intFirstTool) Then
        ' Select the first tool
        Call SetCurrentTool(intFirstTool)
        While (IsToolChanging())
            Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
        Wend
    End If

    ' Save the tool length compensation value for the first tool
    ' if the tool length compensation is enabled
    Dim dblFirstToolComp As Double
    dblFirstToolComp = TernaryIf(SAVED_EN_TOOL_LENGTH, GetToolParam(intFirstTool, TOOL_Z_OFFSET), 0.0)

    ' Ask user for the second tool number
    Dim intSecondTool As Integer
    intSecondTool = BoundedIntQuestion("Enter the second tool number (1-255):", 1, 255)

    ' Disable tool length compensation
    Call SetToolLengthEnabled(False)
```

```
439
440         ' If ENABLE_PROBE_CURR_POS state is set to True, it will
441         ' propose the user to send the machine to home position.
442         ' Otherwise, it will send the machine to home position
443         ' and safe Z position by default.
444         '
445         ' If the user presses OK, it will automatically send the
446         ' machine to home position, otherwise the machine will
447         ' be sent to only safe Z position.
448         If (ENABLE_PROBE_CURR_POS) Then
449             ' Propose the user to send the machine to home position
450             ' or start probing from the current XY position
451             Call ProposeSendHome()
452         Else
453             ' Send the machine to home position
454             Call MachMSG("Sends the machine to home position (G28)", "Auto Tool Zero", MSG_TYPE_OK)
455             Call SendHome()
456         End If
457
458         ' Ask the user for a confirmation to start probing
459         Dim blnAnswerProbe As Integer
460         blnAnswerProbe = MachMSG("Start probing the Z-axis?" & CRLF & CRLF & _
461             "Press Yes to continue or No to cancel.", _
462             "Auto Tool Zero", MSG_TYPE_YES_NO)
463
464         ' Start probing if the user pressed Yes
465         If (blnAnswerProbe = MSG_RETURN_YES) Then
466             ' Start probing with the first tool
467             Dim objFirstProbeResult As Object
468             Set objFirstProbeResult = StartProbing()
469
470             If (Not objFirstProbeResult(KEY_PROBE_EXIT)) Then
471                 ' Save the first tool position
472                 Dim dblFirstToolPos As Double
473                 dblFirstToolPos = objFirstProbeResult(KEY_PROBE_POS)
474
475                 ' Send the machine to safe Z position
476                 Call SendSafeZ(Z_UPPER_BOUND)
477
478                 ' Propose the user to change the tool
479                 Dim blnToolChange As Boolean
480                 blnToolChange = ProposeToolChange()
481
482                 If (blnToolChange) Then
483                     ' Set the second tool as the current tool
484                     Call SetCurrentTool(intSecondTool)
485                     While (IsToolChanging())
486                         Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
487                     Wend
488
489                     ' Start probing with the second tool
490                     Dim objSecondProbeResult As Object
491                     Set objSecondProbeResult = StartProbing()
492
493                     If (Not objSecondProbeResult(KEY_PROBE_EXIT)) Then
494                         ' Save the second tool position
495                         Dim dblSecondToolPos As Double
496                         dblSecondToolPos = objSecondProbeResult(KEY_PROBE_POS)
497
498                         ' Calculate tool compensation with the difference between
499                         ' the two tool lengths and the first tool length compensation
500                         '
501                         ' The formula is: ToolCompensation = SecondToolPos - FirstToolPos + FirstToolComp
502                         Dim dblToolComp As Double
503                         dblToolComp = roun(dblSecondToolPos - dblFirstToolPos + dblFirstToolComp)
504
505                         ' Ask user for a confirmation to set the tool length compensation
506                         Dim blnAnswerSet As Integer
507                         blnAnswerSet = MachMSG("Tool length compensation: " & dblToolComp & STR_MACRO_UNITS & CRLF & CRLF & _
508                             "Press OK to continue or Cancel to deny the compensation.", _
509                             "Auto Tool Zero", MSG_TYPE_OK_CANCEL)
510
511                         ' Debug the answer
512                         Call Debug(MACRO_MSG_TYPE_STATUS, "Return message: " & blnAnswerSet)
```

```
                    If (blnAnswerSet = MSG_RETURN_OK) Then
                        ' Set the tool length compensation for the second tool
                        ' if the user confirms the compensation and enable
                        ' the tool length compensation in the machine
                        Call SetToolParam(intSecondTool, TOOL_Z_OFFSET, dblToolComp)
                        SAVED_EN_TOOL_LENGTH = True
                    Else
                        ' Cancel tool length compensation and switch back to
                        ' the first tool if the user denies the compensation
                        Call Message("Tool length compensation canceled")
                        Call SetCurrentTool(intFirstTool)
                        While (IsToolChanging())
                            Call Sleep(100) ' Sleep, so other threads can run while we're waiting
                        Wend
                    End If
                End If
            End If
        End If

        ' Send the machine to safe Z position
        Call SendSafeZ(Z_UPPER_BOUND)
    End If

    ' Propose the user to return to the saved position
    If (ENABLE_RETURN_TO_SAVED_POS) Then
        Call ProposeReturnToSavedPos()
    End If
End Sub

' ------------------------------------------- '
'          Machine-related subroutines        '
' ------------------------------------------- '

' Returns the current move mode (G00/G01).
'
' It will return 0 for rapid move (G00)
' and 1 for linear move (G01).
Function GetMoveMode() As Integer
    GetMoveMode = GetOEMDRO(DRO_MOVE_MODE)
End Function

' Sets the move mode (G00/G01)
Sub SetMoveMode(ByVal MoveMode As Integer)
    Select Case MoveMode
        Case MOVE_MODE_RAPID To MOVE_MODE_LINEAR
            Call Code("G" & MoveMode)
        Case Else
            Call Message("Invalid move mode (" & MoveMode & ") Setting to G01")
            Call Code("G01")
    End Select
End Sub

' Returns the current machine units (G20/G21).
'
' It will return 0 for millimeters (G21)
' and 1 for inches (G20).
Function GetUnits() As Integer
    GetUnits = GetParam("Units")
End Function

' Sets the machine units (in/mm) (G20/G21)
Sub SetUnits(ByVal Units As Integer)
    Select Case Units
        Case UNITS_MM
            Call Code("G21")
        Case UNITS_IN
            Call Code("G20")
        Case Else
            Call Message("Invalid units (" & Units & ") Setting to G21")
            Call Code("G21")
    End Select
End Sub
```

```
' Returns whether the tool length compensation is enabled or not.
'
' It will return True if the tool length compensation is enabled
' and False if the tool length compensation is disabled.
Function IsToolLengthEnabled() As Boolean
    IsToolLengthEnabled = GetOEMLED(LED_TOOL_LENGTH)
End Function

' Sets the tool length compensation state.
Sub SetToolLengthEnabled(ByVal Enabled As Boolean)
    If (Enabled) Then
        Call Code("G43")
    Else
        Call Code("G49")
    End If
End Sub

' Returns wether the tool is changing or not.
'
' It will return True if the tool is changing
' and False if the tool is not changing.
Function IsToolChanging() As Boolean
    IsToolChanging = GetOEMLED(LED_TOOL_CHANGING)
End Function

' Returns the current distance mode (G90/G91).
'
' It will return 0 for absolute distance mode (G90)
' and 1 for incremental distance mode (G91).
Function GetDistMode() As Integer
    If (GetOEMLed(LED_COORD_ABS) = True) Then
        GetDistMode = DIST_MODE_ABS
    ElseIf (GetOEMLed(LED_COORD_INC) = True) Then
        GetDistMode = DIST_MODE_INC
    End If
End Function

' Sets the distance mode (G90/G91)
Sub SetDistMode(ByVal DistMode As Integer)
    Select Case DistMode
        Case DIST_MODE_ABS To DIST_MODE_INC
            Call Code("G9" & DistMode)
        Case Else
            Call Message("Invalid distance mode (" & DistMode & ") Setting to G90")
            Call Code("G90")
    End Select
End Sub

' Returns offset for the current fixture by axis.
Function GetFixtureOffset(ByVal Axis As Integer) As Double
    Select Case Axis
        Case X_AXIS To C_AXIS
            GetFixtureOffset = GetOEMDRO(DRO_FIXTURE_OFFSET_X + Axis)
        Case Else
            Call Message("Invalid axis (" & Axis & ")")
            GetFixtureOffset = 0.0
    End Select
End Function

' Sets the fixture offset for the current fixture by axis.
Sub SetFixtureOffset(ByVal Axis As Integer, ByVal Offset As Double)
    Select Case Axis
        Case X_AXIS To C_AXIS
            Call SetOEMDRO(DRO_FIXTURE_OFFSET_X + Axis, Offset)
        Case Else
            Call Message("Invalid axis (" & Axis & ")")
    End Select
End Sub

' Returns whether the probe is wired or not
'
' It will return True if the probe is wired
' and False if the probe is not wired
Function IsProbeWired() As Boolean
```

```
    IsProbeWired = GetOEMLed(LED_PROBE)
End Function

' Proposes the user to change the tool.
'
' It will display a message box asking the user to change the tool.
' If the user presses Yes, it will ask for confirmation.
' If the user confirms, the function returns True.
' If the user presses No at any point, the question will repeat.
' If the user presses Cancel at any point, the macro will exit.
Function ProposeToolChange() As Boolean
    Dim intReturn As Integer
    Dim intConfirm As Integer

    ' Loop until a definitive decision (Yes or Cancel) is made
    Do
        intReturn = MachMSG("Change the tool and press OK to continue.", _
            "Auto Tool Zero", MSG_TYPE_OK_CANCEL)

        If (intReturn = MSG_RETURN_OK) Then
            ' User pressed OK, ask for confirmation
            intConfirm = MachMSG("Are you sure you have changed the tool?" & CRLF & CRLF & _
                "If you press No, the macro will repeat the question.", _
                "Auto Tool Zero", MSG_TYPE_YES_NO_CANCEL)

            Select Case intConfirm
                Case MSG_RETURN_CANCEL
                    ' User pressed Cancel, exit the macro immediately
                    ProposeToolChange = False
                    Exit Do
                Case MSG_RETURN_NO
                    ' User chose No, loop will repeat (implicit)
                Case MSG_RETURN_YES
                    ProposeToolChange = True
                    Exit Do
            End Select
        Else
            ' User pressed Cancel in the initial prompt, exit the macro immediately
            ProposeToolChange = False
            Exit Do
        End If
    Loop
End Function

' Proposes the user to send the machine to home position.
'
' It will display a message box with the message and
' wait for the user to press Yes or No. If the user
' presses Yes, it will send the machine to home position
' and return True. If the user presses No, it will
' send the machine to safe Z position and return False.
Function ProposeSendHome() As Boolean
    Dim intReturn As Integer

    ' Loop until a definitive decision (Yes or No with confirmation) is made
    Do
        intReturn = MachMSG("Send the machine to home position (G28) and safe Z?" & CRLF & CRLF & _
            "If you press No, the probe will be started from the current XY position and safe Z.", _
            "Auto Tool Zero", MSG_TYPE_YES_NO)

        If (intReturn = MSG_RETURN_YES) Then
            Call SendHome()
            ProposeSendHome = True
            Exit Do
        Else
            Dim intConfirm As Integer
            intConfirm = MachMSG("Are you sure you want to continue from the current position?", _
                "Auto Tool Zero", MSG_TYPE_YES_NO)

            If (intConfirm = MSG_RETURN_YES) Then
                Call SendSafeZ(Z_UPPER_BOUND)
                ProposeSendHome = False
                Exit Do
            End If
```

```
        End If
    Loop
End Function

' Proposes the user to return to the saved position.
'
' It will display a message box with the message and
' wait for the user to press Yes or No. If the user
' presses Yes, it will return to the saved position
' and return True. If the user presses No, it will
' stay in the current position and return False.
Function ProposeReturnToSavedPos() As Boolean
    ' If the machine is already in the saved position, do not propose to return
    If (GetABSPosition(Z_AXIS) = SAVED_Z And _
        GetABSPosition(X_AXIS) = SAVED_X And _
        GetABSPosition(Y_AXIS) = SAVED_Y) Then
        ProposeReturnToSavedPos = False
        Exit Function
    End If

    Dim intReturn As Integer

    ' Propose the user to return to the saved position
    intReturn = MachMSG("Return to the saved position?" & CRLF & CRLF & _
        "If you press No, the machine will stay in the current position.", _
        "Auto Tool Zero", MSG_TYPE_YES_NO)

    If (intReturn = MSG_RETURN_YES) Then
        intReturn = MachMSG("Are you sure you want to return to the saved position?", _
            "Auto Tool Zero", MSG_TYPE_YES_NO)

        If (intReturn = MSG_RETURN_YES) Then
            Call ReturnToSavedPos()
            ProposeReturnToSavedPos = True
        End If
    End If

    ProposeReturnToSavedPos = False
End Function

' Sends the machine to the home position and safe Z
Sub SendHome()
    ' Sleep for 1 seconds
    Call Sleep(1000)

    ' Set the params to macro parameters
    Call SetUnits(MACRO_UNITS)
    Call SetDistMode(DIST_MODE_ABS)
    Call SetToolLengthEnabled(False)
    While (IsMoving())
        Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
    Wend

    ' Set the params to macro parameters
    Call Debug(MACRO_MSG_TYPE_STATUS, "Moving to the safe position for Z-axis in machine coordinates (" & Z_UPPER_BOUND & ")")

    ' Move to the safe position for Z-axis
    Call Code("G53 G0 Z" & Z_UPPER_BOUND)
    While (IsMoving())
        Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
    Wend

    Call Debug(MACRO_MSG_TYPE_STATUS, "Moving to the machine home for X and Y axes")

    ' Move to the home for x and y axes
    Call Code("G91 G28 X0 Y0")
    While (IsMoving())
        Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
    Wend

    Call Debug(MACRO_MSG_TYPE_STATUS, "Moved to the machine home for X and Y axes and safe Z position")

    Call SetDistMode(DIST_MODE_ABS)
End Sub
```

```
809
810 ' Returns the machine to the saved position
811 Sub ReturnToSavedPos()
812     ' Sleep for 1 seconds
813     Call Sleep(1000)
814
815     ' Set the params to macro parameters
816     Call SetUnits(SAVED_UNITS)
817     Call SetDistMode(DIST_MODE_ABS)
818     While (IsMoving())
819         Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
820     Wend
821
822     ' Restore the original tool length compensation state
823     Call SetToolLengthEnabled(SAVED_EN_TOOL_LENGTH)
824
825     ' Move to the saved position for Z-axis
826     Call Debug(MACRO_MSG_TYPE_STATUS, "Returning to the saved position for Z-axis in machine coordinates")
827     Call Code("G53 G0 Z" & SAVED_Z)
828     While (IsMoving())
829         Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
830     Wend
831
832     ' Move to the saved position for X and Y axes
833     Call Debug(MACRO_MSG_TYPE_STATUS, "Returning to the saved position for X and Y axes in machine coordinates")
834     Call Code("G53 G0 X" & SAVED_X & " Y" & SAVED_Y)
835     While (IsMoving())
836         Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
837     Wend
838 End Sub
839
840 ' Probes the Z-axis of the machine and returns the
841 ' position of the probe in machine coordinates.
842 Function StartProbing() As Object
843     Call Sleep(1000)
844
845     ' Set the params to macro parameters
846     Call SetMoveMode(MOVE_MODE_LINEAR)
847     Call SetUnits(MACRO_UNITS)
848     Call SetDistMode(DIST_MODE_ABS)
849     Call SetToolLengthEnabled(False)
850     While (IsMoving())
851         Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
852     Wend
853
854     Dim dblProbePos As Double
855     Dim intResponse As Integer
856     Dim blnExitState As Boolean
857
858     Do
859         ' Perform fast probing
860         Call Message("Fast probing...")
861         Call Code("G31 Z" & Z_LOWER_BOUND & " F" & PROBE_FEED)
862         While (IsMoving())
863             Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
864         Wend
865
866         ' Send the machine to safe Z position after fast probe
867         ' (Formula: Safe Z = Current Z + SAFE_MARGIN)
868         Call SendSafeZ(GetABSPosition(Z_AXIS))
869
870         ' Perform finish probing
871         Call Message("Finishing probing...")
872         Call Code("G31 Z" & Z_LOWER_BOUND & " F" & FINISH_PROBE_FEED)
873         While (IsMoving())
874             Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
875         Wend
876
877         ' Write the probe position to the variable
878         dblProbePos = GetVar(VAR_PROBE_OFFSET_Z)
879
880         ' Send the machine to safe Z position after finish probe
881         Call Message("Probe finished.")
882         Call SendSafeZ(GetABSPosition(Z_AXIS))
```

```
883
884         ' Display probe result and ask for action
885         intResponse = MachMSG("Probe position: " & roun(dblProbePos) & STR_MACRO_UNITS & CRLF & CRLF & _
886             "Press Try Again to re-probe, Continue to accept, or Cancel to exit the macro.", _
887             "Probe Result", MSG_TYPE_CANCEL_TRY_CONTINUE)
888
889         Select Case intResponse
890             Case MSG_RETURN_TRY
891                 ' Continue loop to re-probe
892                 Call SendSafeZ(Z_UPPER_BOUND)
893                 Call Message("Re-probing...")
894             Case MSG_RETURN_CONTINUE
895                 ' User accepts the probe, exit loop
896                 blnExitState = False
897                 Exit Do
898             Case MSG_RETURN_CANCEL
899                 ' User cancels, exit macro
900                 blnExitState = True
901                 Exit Do
902             Case Else
903                 ' Unexpected response, also exit as an error
904                 Call ExitWithError(MACRO_MSG_TYPE_STATUS, "Unexpected response during probing. Exiting.")
905                 blnExitState = True
906                 Exit Do
907         End Select
908     Loop
909
910     ' Build the probe result object
911     Dim objProbeResult As Object
912     Set objProbeResult = CreateObject("Scripting.Dictionary")
913     objProbeResult.Add KEY_PROBE_EXIT, blnExitState
914     objProbeResult.Add KEY_PROBE_POS, dblProbePos
915
916     ' Return the probe result
917     StartProbing = objProbeResult
918 End Function
919
920 ' Sends the machine to safe z position
921 Sub SendSafeZ(ByVal ZPos As Double)
922     Dim dblSafePos As Double
923     dblSafePos = ZPos + SAFE_MARGIN
924
925     Call Code("G53 G0 Z" & TernaryIf(dblSafePos > Z_UPPER_BOUND, Z_UPPER_BOUND, dblSafePos))
926     While (IsMoving())
927         Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
928     Wend
929 End Sub
930
931 ' ------------------------------------------- '
932 '             Macro-related section           '
933 ' ------------------------------------------- '
934
935 ' This subroutine is used for debugging purposes.
936 '
937 ' It will display the message with "DEBUG:" prefix
938 ' in the status bar of Mach3 or in a dialog box.
939 Sub Debug(ByVal MsgType As Integer, ByVal MsgText As String)
940     If (ENABLE_DEBUG_MODE) Then
941         Select Case MsgType
942             Case MACRO_MSG_TYPE_STATUS
943                 Call Message("DEBUG: " & MsgText)
944             Case MACRO_MSG_TYPE_DIALOG
945                 Call CancellableMsg("DEBUG: " & MsgText, "Debug Message")
946             Case Else
947                 Call ExitWithError(MACRO_MSG_TYPE_DIALOG, "Invalid message type")
948         End Select
949     End If
950 End Sub
951
952 ' This function is used to display a message box with
953 ' an input field for the user to enter a value.
954 '
955 ' It will repeat the question until the user enters a valid
956 ' value. The value must be a number and must be within the
```

```
' specified bounds (MinValue and MaxValue).
Function BoundedIntQuestion(ByVal QuestionText As String, ByVal MinValue As Integer, ByVal MaxValue As Integer) As Integer
    BoundedIntQuestion = CInt(BoundedDoubleQuestion(QuestionText, MinValue, MaxValue))
End Function

' This function is used to display a message box with
' an input field for the user to enter a value.
'
' It will repeat the question until the user enters a valid
' value. The value must be a number.
Function IntQuestion(ByVal QuestionText As String) As Integer
    IntQuestion = CInt(DoubleQuestion(QuestionText))
End Function

' This function is used to display a message box with
' an input field for the user to enter a value.
'
' It will repeat the question until the user enters a valid
' value. The value must be a number and must be within the
' specified bounds (MinValue and MaxValue).
Function BoundedDoubleQuestion(ByVal QuestionText As String, ByVal MinValue As Double, ByVal MaxValue As Double) As Double
    Dim strAnswer As String
    Dim dblAnswer As Double

    Do
        strAnswer = Question(QuestionText)

        If (IsNumeric(strAnswer)) Then
            dblAnswer = CDbl(strAnswer)

            If (dblAnswer >= MinValue And dblAnswer <= MaxValue) Then
                BoundedDoubleQuestion = dblAnswer
                Exit Do
            Else
                Call MachMSG("Invalid input. Please enter a number between " & MinValue & " and " & MaxValue & ".", "Error", MSG_
TYPE_OK)
            End If
        Else
            Call MachMSG("Invalid input. Please enter a number.", "Error", MSG_TYPE_OK)
        End If
    Loop
End Function

' This function is used to display a message box with
' an input field for the user to enter a value.
'
' It will repeat the question until the user enters a valid
' value. The value must be a number.
Function DoubleQuestion(ByVal QuestionText As String) As Double
    Dim strAnswer As String

    Do
        strAnswer = Question(QuestionText)

        If (IsNumeric(strAnswer)) Then
            DoubleQuestion = CDbl(strAnswer)
            Exit Do
        Else
            Call MachMSG("Invalid input. Please enter a number.", "Error", MSG_TYPE_OK)
        End If
    Loop
End Function

' This subroutine is used to display a message box with
' an OK and Cancel button.
'
' It will display the message and wait for the user to
' press OK or Cancel. If the user presses Cancel, it will
' exit the macro.
Sub CancellableMsg(ByVal Text As String, ByVal Title As String)
    Dim intReturn As Integer
    intReturn = MachMSG(Text & CRLF & CRLF & _
        "Press OK to continue or Cancel to exit the macro.", _
        Title, MSG_TYPE_OK_CANCEL)
```

```
    ' If the user presses Cancel, it will exit the macro
    ' and restore the original parameters
    If (intReturn = MSG_RETURN_CANCEL) Then
        Call ExitMacro()
    End If
End Sub

' This subroutine is called when the macro encounters an error.
'
' It will display the error message, restore the original properties
' and end the macro.
Sub ExitWithError(ByVal MsgType As Integer, ByVal MsgText As String)
    Dim Msg As String
    Msg = "ERROR: " & MsgText

    Select Case MsgType
        Case MACRO_MSG_TYPE_STATUS
            Call Message(Msg)
        Case MACRO_MSG_TYPE_DIALOG
            Call MachMSG(Msg & CRLF & CRLF & _
                "Press OK to exit the macro.", _
                "Critical Error", MSG_TYPE_OK)
        Case Else
            Call ExitWithError(MACRO_MSG_TYPE_DIALOG, "Invalid message type")
    End Select

    Call ExitMacro()
End Sub

' This subroutine is called when the macro is finished.
'
' It will restore the original properties and move to
' the home position.
Sub ExitMacro()
    ' Restore original properties
    Call SetMoveMode(SAVED_MOVE_MODE)               ' Restore original move mode
    Call SetUnits(SAVED_UNITS)                      ' Restore original units
    Call SetToolLengthEnabled(SAVED_EN_TOOL_LENGTH) ' Restore original tool length compensation state
    Call SetDistMode(SAVED_DISTANCE_MODE)           ' Restore original coordinate mode
    Call SetFeedRate(SAVED_FEED / 60)               ' Restore original feedrate

    While (IsMoving())
        Call Sleep(100) ' Sleep, so other threads can run while we’re waiting
    Wend

    Call Message("Exited macro and restored original properties")
    End
End Sub

' This is a simple implementation of the ternary operator.
'
' It takes a condition and two values, and returns the
' first value if the condition is true, and the second
' value if the condition is false.
Function TernaryIf(ByVal Condition As Boolean, ByVal TrueValue As Variant, ByVal FalseValue As Variant) As Variant
    If (Condition) Then
        TernaryIf = TrueValue
    Else
        TernaryIf = FalseValue
    End If
End Function
```